ADVANCED SQL

Part 1: Theory Questions

Q1. What is a Common Table Expression (CTE), and how does it improve SQL query readability?

A CTE is a temporary result set that you can reference within another SELECT, INSERT, UPDATE, or DELETE statement. It is defined using the WITH clause.
Readability: It allows you to break down complex, nested queries into logical blocks. Instead of deep subqueries, you name each block, making the final query read like a step-by-step narrative.

Q2. Why are some views updatable while others are read-only? Explain with an example.
A view is updatable if the database can map the change directly back to a single row in the underlying table.

Updatable: A simple view selecting columns from one table without modifications.
Read-only: Views that use aggregations (SUM, AVG), DISTINCT, GROUP BY, or JOINs (in some systems).
Example: A view showing TotalSales per product using SUM(Quantity) is read-only because the database wouldn't know which specific sale record to change if you tried to update the total.

Q3. What advantages do stored procedures offer compared to writing raw SQL queries repeatedly?

Maintainability: You update the logic in one place rather than in every application script.
Performance: Procedures are compiled and cached by the server, reducing execution time.
Security: You can give users permission to execute a procedure without giving them direct access to the underlying tables.
Reduced Network Traffic: Only the procedure call is sent over the network, not the entire multi-line SQL script.

Q4. What is the purpose of triggers in a database? Mention one use case where a trigger is essential.

Triggers are special stored programs that automatically execute (or "fire") in response to specific events (INSERT, UPDATE, DELETE) on a table.
Essential Use Case: Auditing. If you need to keep a record of every time a price is changed in the Products table, a trigger can automatically copy the old price, new price, and timestamp into an AuditTable.

Q5. Explain the need for data modelling and normalization when designing a database.
Data Modelling: Acts as a blueprint to ensure all business requirements and relationships are captured before building.

Normalization: The process of organizing data to reduce redundancy (saving space) and improve data integrity (preventing anomalies where data is updated in one place but forgotten in another).
Q6. Write a CTE to calculate the total revenue for each product (Revenues = Price × Quantity), and return only products where revenue > 3000.

Q6. Write a CTE to calculate the total revenue for each product (Revenue = Price \times Quantity), and return only products where revenue > 3000.

```
61
62      -- 6. Final Q6 answer run karein
63      WITH ProductRevenue AS (
64          SELECT p.ProductName, (p.Price * s.Quantity) AS TotalRevenue
65          FROM Products p
66          JOIN Sales s ON p.ProductID = s.ProductID
67      )
68      SELECT * FROM ProductRevenue
69      WHERE TotalRevenue > 3000;
```

esult Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductName	TotalRevenue
Keyboard	4800.00
Mouse	8000.00
Chair	5000.00
Desk	5500.00

Q7. Create a view named vw_CategorySummary that shows: Category, TotalProducts, AveragePrice.

```
72
73  CREATE VIEW vw_CategorySummary AS
74  SELECT
75      Category,
76      COUNT(ProductID) AS TotalProducts,
77      AVG(Price) AS AveragePrice
78  FROM Products
79  GROUP BY Category;
80  SELECT * FROM vw_CategorySummary;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Category	TotalProducts	AveragePrice
Electronics	2	1050.000000
Furniture	2	4000.000000

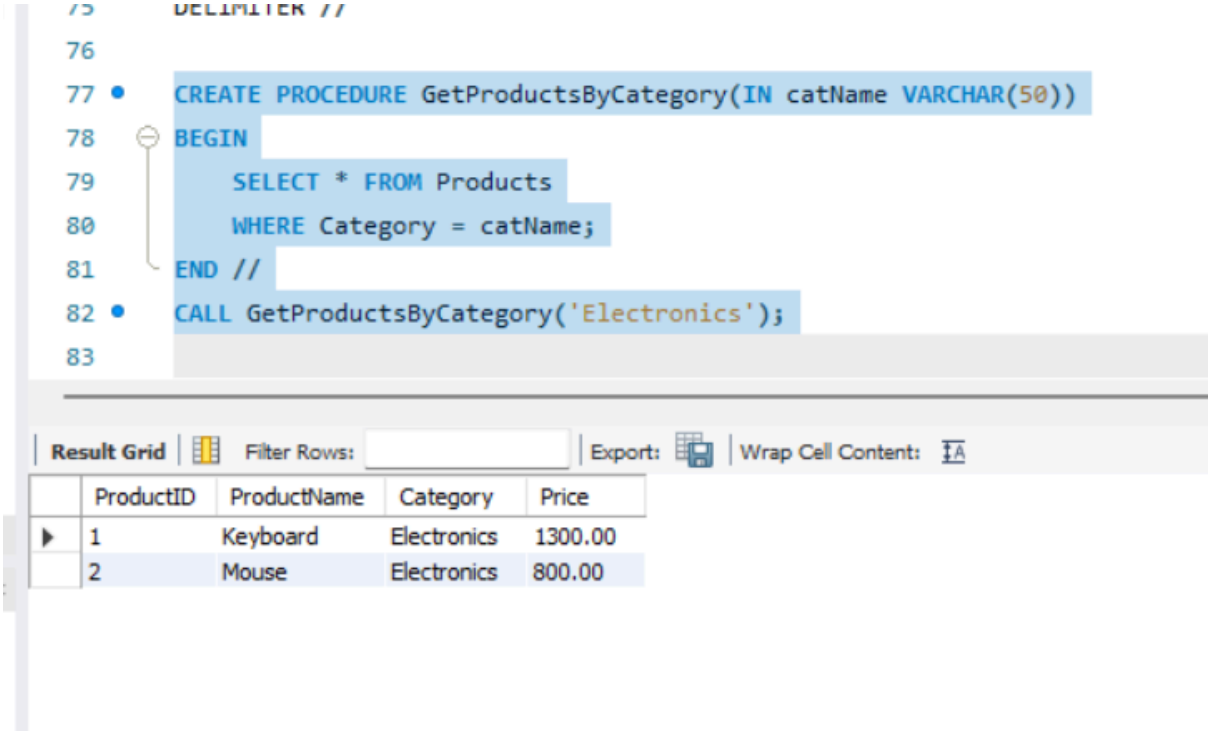
Q8. Create an updatable view containing ProductID, ProductName, and Price. Then update the price of ProductID = 1 using the view.

```
Limit to 1000 rows
6  CREATE VIEW vw_ProductPricing AS
7  SELECT ProductID, ProductName, Price
8  FROM Products;
9
0  -- Update the view
1  UPDATE vw_ProductPricing
2  SET Price = 1300
3  WHERE ProductID = 1;
4  SELECT * FROM Products;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

ProductID	ProductName	Category	Price
1	Keyboard	Electronics	1300.00
2	Mouse	Electronics	800.00
3	Chair	Furniture	2500.00
4	Desk	Furniture	5500.00
NULL	NULL	NULL	NULL

Q9. Create a stored procedure that accepts a category name and returns all products belonging to that category.



```
DELIMITER //

CREATE PROCEDURE GetProductsByCategory(IN catName VARCHAR(50))
BEGIN
    SELECT * FROM Products
    WHERE Category = catName;
END //
CALL GetProductsByCategory('Electronics');
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductID	ProductName	Category	Price
1	Keyboard	Electronics	1300.00
2	Mouse	Electronics	800.00